Stock Trading RL Agent

Sarthak Nimesh Yadav (UBIT: syadav8)
CSE 4/546: Reinforcement Learning

Contents

1 Introduction **2**

2 Data Acquisition & Pre-Processing **2**
2.1 Market Data 2
2.2 Technical Indicators 3
2.3 Fundamental and Sentiment Signals 3

3 RL Environment **4**

4 RL Algorithms **4**
4.1 REINFORCE 4
4.2 Advantage Actor–Critic (A2C) 5
4.3 Proximal Policy Optimization (PPO) . . . . . .
6
4.4 Deep Deterministic Policy Gradient (DDPG) . . . . . .
7 **9**

6 Results & Analysis **9**
6.1 Plots 10
6.2 Comparative Discussion 10

7 Conclusion Future Work **11**

8 References **11**

Abstract

In this project, we develop a reinforcement learning agent to trade the 30 constituents of the Dow Jones Industrial Average. Our agent's state representation fuses **technical indicators**, **fundamental factors**, and **sentiment scores** into a single input vector. We implement four deep RL algorithms REINFORCE, A2C, PPO, and DDPG from scratch in PyTorch and evaluate them over the 2016–2019 period against the DJI benchmark. Our main contributions are:

1. Designing a unified, multi-signal state representation.
2. Building end-to-end implementations of four policy-based agents.
3. Conducting an ablation study to quantify the impact of sentiment features.

Our experiments show that DDPG yields the highest Sharpe ratio, and that incorporating sentiment data consistently improves risk-adjusted performance across all agents.

1 Introduction

In recent years, **algorithmic trading** has grown rapidly as financial firms and individual investors seek more automated and scalable ways to execute buy and sell decisions. Traditional rule-based systems rely on fixed heuristics, whereas **Reinforcement Learning (RL)** enables an agent to discover its own strategies by interacting with market data and optimizing for long-term returns. While RL has achieved remarkable success in domains such as games and robotics, its application to real-world trading presents unique challenges.

Financial markets are inherently noisy and non-stationary, and trading strategies must account for transaction costs, slippage, and changing market dynamics. Moreover, most prior work concentrates on a single source of information typically price history or a handful of technical indicators while overlooking other valuable signals like news sentiment.

In this project, we implement four deep RL agents REINFORCE, A2C, PPO, and DDPG from scratch in PyTorch, and we introduce a unified framework that fuses market prices, technical indicators, and **FinBERT**–based sentiment scores into each agent's state representation. Our goal is to understand how combining these diverse signals affects the agents' ability to learn profitable and robust trading policies.

Our key contributions are:

- A streamlined data pipeline for downloading and preprocessing **Dow 30** price data, computing ten standard technical indicators, and scoring news headlines with **FinBERT**.
- End-to-end implementations of four policy-based and actor-critic algorithms using a consistent network backbone to ensure a fair comparison.
- An empirical study demonstrating that agents leveraging sentiment features consistently outperform those using only price and indicator inputs in terms of risk-adjusted returns.

2 Data Acquisition & Pre-Processing

To train robust reinforcement learning agents, we assemble and preprocess three complementary data sources: historical market quotes, derived technical indicators, and news-based sentiment scores.

2.1 Market Data

We use the **FinRL** library's integrated downloader, which interfaces with the Yahoo Finance API [1]. We retrieve daily **Open**, **High**, **Low**, **Close**, and **Volume** (**OHLCV**) data for all 30 constituents of the Dow Jones Industrial Average. We split our dataset into a three-year training window (`2016-01-01` to `2019-01-01`) and a six-month testing window (`2019-01-02` to `2019-07-01`). This division ensures that our agents experience a variety of market conditions during training and are then evaluated on truly out-of-sample data. The raw OHLCV series form the basis for all subsequent feature engineering steps and state construction.

2.2 Technical Indicators

Technical indicators are rule-based signals derived from historical price and volume data that help capture market momentum, volatility, and trends. We use the **FinRL** library's integration with the **stockstats** package to compute ten daily indicators for each stock, along with two market-wide metrics (`vix` and `turbulence`). Our selected indicators are:

- `macd`: the difference between the 12-day and 26-day exponential moving averages, capturing momentum shifts.
- `boll_ub` / `boll_lb`: the upper and lower Bollinger Bands set at two standard deviations above and below a 20-day simple moving average, highlighting volatility extremes.
- `rsi_30`: the 30-day Relative Strength Index, measuring recent gains relative to losses to flag overbought or oversold conditions.
- `cci_30`: the 30-day Commodity Channel Index, comparing price deviations from the 30-day moving average to assess cyclical trends.
- `dx_30`: the 30-day Directional Movement Index, quantifying trend strength.
- `close_30_sma` / `close_60_sma`: the 30-day and 60-day simple moving averages of closing prices, indicating short- and medium-term trends.
- `vix`: the CBOE Volatility Index, serving as a proxy for overall market risk sentiment.
- `turbulence`: FinRL's turbulence index, estimating market instability based on cross-asset return correlations.

2.3 Fundamental and Sentiment Signals

To enrich our state representation beyond price and volume, we incorporate a news-based sentiment feature using the **Massive Stock News Analysis DB for NLP Backtests** dataset from Kaggle [2]. We follow these steps:

1. Load the raw CSV and inspect the key columns: `date`, `tic`, and `headline`.
2. Clean each headline by converting to lowercase and removing non-alphanumeric characters.
3. Filter the dataset:
   (a) Retain only rows where `tic` matches one of the 30 DJI constituents.
   (b) Restrict dates to our training period (`2016-01-01` to `2019-01-01`) and testing period (`2019-01-02` to `2019-07-01`).

Next, we tokenize each cleaned headline and feed it into a pretrained **FinBERT** model fine-tuned on financial text to obtain a probability distribution over *positive*, *neutral*, and *negative* classes. We compute a **sentiment score** by subtracting the negative probability from the positive probability (yielding a value in $[-1, 1]$). To produce a daily feature for each stock, we average all headline scores for that ticker on each date.

Finally, we merge these daily sentiment scores into our OHLCV plus technical-indicator dataframe by aligning on `date` and `tic`. We assign a neutral score of zero for days with no news and forward-fill any remaining gaps to maintain continuity. The result is a unified dataset, ready for input into our RL environment, where each row contains market prices, technical features, and a sentiment signal.

3 RL Environment

Our reinforcement learning environment defines how our agent interacts with market data and executes trades. We build on the `StockTradingEnv` class from the FinRL library and extend it to include our multi-modal state inputs.

- **State Representation:** At each daily time step, our agent observes a flattened, normalized vector that includes:
  - The current cash balance in the portfolio.
  - Normalized closing prices for each of the 30 Dow Jones constituents.
  - Values of our ten technical indicators for each stock (e.g., MACD, RSI).
  - A daily sentiment score per stock derived from FinBERT analysis.
  - The number of shares held for each stock.

  This comprehensive snapshot helps the agent understand both market conditions and its own position before choosing an action.

- **Action Space:** We use a continuous action space via `gym.spaces.Box` with dimension equal to the number of stocks (30). Each component of the action vector represents the proportion of capital to buy or sell for the corresponding stock, typically ranging from –1 (sell maximum) to +1 (buy maximum), and is scaled by a predefined trade limit (`hmax`).

- **Reward Function:** To encourage profitable but prudent trading, we define the instantaneous reward as the change in total portfolio value minus transaction costs, scaled for numerical stability:

$$R_t = \text{scale} \times \big[(V_t - V_{t-1}) - C_t\big] \tag{1}$$

  where:
  - V_t is the total portfolio value at time t (cash + market value of holdings).
  - V_{t-1} is the portfolio value at the previous time step.
  - C_t is the transaction cost incurred at time t (based on buy/sell cost percentages).
  - `scale` (e.g., 1×10^{-4}) ensures stable training.

  This reward structure guides the agent to increase portfolio value while penalizing excessive trading.

4 RL Algorithms

We implemented four different RL algorithms, representing various approaches within the field, to compare their effectiveness on our stock trading task. Each algorithm was implemented in Python using PyTorch for neural network components.

4.1 REINFORCE

REINFORCE is one of the simplest Monte Carlo policy-gradient methods [4]. In our project, we adapt it to handle continuous actions by parameterizing a Gaussian policy. We summarize our approach below:

1. **Network Architecture.** We use a single policy network that takes the current state vector (normalized features) as input and outputs the parameters of a multivariate Gaussian distribution over actions:
   - The input passes through two fully connected hidden layers (each with 64 units and ReLU activations).
   - We then branch into two heads: one head produces the mean vector $\mu_\theta(s)$, and the other produces a shared learnable log-standard-deviation $\log \sigma$. We exponentiate $\log \sigma$ to get σ.
   - Together, these define our policy $\pi_\theta(a|s) = \mathcal{N}\big(\mu_\theta(s),\, \sigma^2\big)$.
   - At each step, we sample $a \sim \pi_\theta(\cdot|s)$ and clamp it to the environment's action bounds.

2. **Policy Update.** We collect full episodes (trajectories) and update our network parameters to maximize the expected return:

$$J(\theta) = E_{\tau\sim\pi_\theta}[\,G_0\,], \quad G_0 = \sum_{t=0}^{T-1} \gamma^t R_{t+1}.$$

We estimate the policy gradient as

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T_i-1} \nabla_\theta \log \pi_\theta(a_{i,t}|s_{i,t}) \big(G_{i,t} - b(s_{i,t})\big).$$

Here,

- $G_{i,t}$ is the discounted return-to-go from step t.
- We use a baseline $b(s)$ equal to the mean return-to-go (standardized across the episode) to reduce variance.
- In practice we minimize the loss $L = -\sum_t \log \pi_\theta(a_t|s_t)\, \widehat{G}_t$, where $\widehat{G}_t$ are the standardized returns.
- We apply the update once per episode using the full trajectory.

3. **Hyperparameters.**

Table 1. Key REINFORCE Hyperparameters

Parameter	Value Used
Learning rate α	5×10^{-4}
Discount factor γ	0.99
Network hidden layers	two layers of 64 units
Optimizer	Adam
Initial log-std deviation	0 (learnable)

4. **Architecture Diagram.** Figure 1 illustrates our policy network's flow from state input to sampled continuous action.

$$\boxed{\text{State} \rightarrow \text{Shared FC–ReLU layers} \rightarrow \left\{\begin{matrix}\text{Mean head}\\ \text{Log-std head}\end{matrix}\right\} \rightarrow \text{Gaussian} \rightarrow \text{Sampled action}}$$

Figure 1. Conceptual architecture of our continuous REINFORCE policy network.

4.2 Advantage Actor–Critic (A2C)

In our project, we implement the Advantage Actor–Critic (A2C) algorithm [5] to handle continuous actions in stock trading. A2C combines a policy network (the Actor) with a value network (the Critic) so that we can compute an advantage estimate $A(s, a)$ to reduce variance in our gradient estimates.

1. **Network Architecture.** We build a single neural network with shared hidden layers that branches into an Actor head and a Critic head:

- *Input:* the state vector s (normalized market features).
- *Shared Layers:* two fully connected layers (64 units each) with ReLU activations.
- *Actor Head:*
  - Outputs the mean vector $\mu_\theta(s)$ (one mean per action dimension).
  - Maintains a learnable log–standard-deviation parameter $\log \sigma$ (shared across dimensions).
  - Defines our policy $\pi_\theta(a|s) = \mathcal{N}\big(\mu_\theta(s),\, \exp(2\log\sigma)\big)$.
- *Critic Head:*

- Outputs a single scalar $V_w(s)$, our estimate of the state value.

2. **Training and Updates.** We collect small batches of experience (e.g. $n = 5$ steps) before updating both networks synchronously:

   - *N–Step Return:*
   $$G_{t:t+n} \;=\; \sum_{k=0}^{n-1} \gamma^k \, R_{t+k+1} \;+\; \gamma^n \, V_w(s_{t+n}) \,.$$
   - *Advantage:*
   $$A_t \;=\; G_{t:t+n} \;-\; V_w(s_t) \,.$$
   - *Actor Loss:*
   $$L_{\text{actor}} \;=\; -\frac{1}{B} \sum_{t \in \text{batch}} \log \pi_\theta(a_t|s_t) \, \big[A_t\big]_{\text{detach}},$$
   where we treat A_t as a constant when backpropagating.
   - *Critic Loss:*
   $$L_{\text{critic}} \;=\; \frac{1}{B} \sum_{t \in \text{batch}} \big(G_{t:t+n} - V_w(s_t)\big)^2.$$
   - *Entropy Bonus:*
   $$L_{\text{entropy}} \;=\; -c_{\text{ent}} \, E\big[H(\pi_\theta(\cdot|s_t))\big]$$
   to encourage exploration.
   - *Total Loss:*
   $$L \;=\; L_{\text{actor}} \;+\; c_{\text{vf}} \, L_{\text{critic}} \;+\; L_{\text{entropy}} \,.$$
   - We apply gradient clipping for stability and perform one update per batch.

3. **Hyperparameters.**

Table 2. Key A2C Hyperparameters

Parameter	Value
Learning rate (lr)	7×10^{-4}
Discount factor (γ)	0.99
N–step horizon (n)	5
Value-loss coefficient (c_{vf})	0.5
Entropy coefficient (c_{ent})	0.01
Network hidden layers	two layers of 64 units
Optimizer	Adam
Gradient clip norm	0.5

4.3 Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) is a modern actor–critic method that offers both stability and strong performance across many continuous control tasks [6]. In our work, we adopt the clipped-surrogate variant of PPO to learn a robust stock-trading policy while preventing overly large policy updates.

1. **Network Architecture.** We use one neural network with shared layers that splits into an Actor head and a Critic head:

   - *Input:* Our state vector s, which contains normalized market features.
   - *Shared Layers:* Two fully connected layers of 64 units each, with ReLU activations.
   - *Actor Head:*
     - Produces the mean vector $\mu_\theta(s)$.
     - Maintains a learnable log-standard-deviation parameter $\log \sigma$, shared across action dimensions.

Thus, our policy is

$$\pi_\theta(a|s) = \mathcal{N}\big(\mu_\theta(s),\, \exp(2\log\sigma)\big).$$

- *Critic Head:* Outputs a scalar value estimate $V_w(s)$.

2. **Data Collection & GAE.** We roll out the current policy $\pi_{\theta_{\text{old}}}$ for 2048 timesteps, storing $\{s_t, a_t, r_t, \log \pi_{\theta_{\text{old}}}(a_t|s_t), V_{w_{\text{old}}}(s_t)\}$. Afterward, we compute advantages A_t^{GAE} via Generalized Advantage Estimation (GAE):

$$\delta_t = r_t + \gamma\, V_{w_{\text{old}}}(s_{t+1}) - V_{w_{\text{old}}}(s_t), \quad A_t^{\text{GAE}} = \sum_{l=0}^{\infty} (\gamma\lambda)^l\, \delta_{t+l}.$$

We then standardize A_t^{GAE} across the batch for numerical stability.

3. **Clipped Surrogate Objective.** We perform $K = 10$ epochs of minibatch updates (batch size 64) over the collected data. Define the probability ratio

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}.$$

The clipped policy objective is

$$L^{\text{CLIP}}(\theta) = E_t\big[\min\big(r_t(\theta)\,\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\,\hat{A}_t\big)\big],$$

with $\epsilon = 0.2$. In code, we minimize $-L^{\text{CLIP}}$.

4. **Value Loss & Total Loss.** Our critic minimizes

$$L^{\text{VF}}(w) = \tfrac{1}{2}\, E_t\big[\big(V_w(s_t) - G_t^{\text{GAE}}\big)^2\big], \quad G_t^{\text{GAE}} = A_t^{\text{GAE}} + V_{w_{\text{old}}}(s_t).$$

We combine losses as

$$L = -L^{\text{CLIP}} + c_1\, L^{\text{VF}}, \quad c_1 = 0.5.$$

After each round of epochs, we set $\theta_{\text{old}} \leftarrow \theta$ and $w_{\text{old}} \leftarrow w$.

5. **Hyperparameters.**

Table 3. Key PPO Hyperparameters

Parameter	Value
Learning rate α	3×10^{-4}
Discount factor γ	0.99
GAE λ	0.95
Clip ϵ	0.2
Epochs (K)	10
Minibatch size	64
Update timesteps	2048
Value coefficient (c_1)	0.5
Entropy coefficient (c_2)	0.0
Hidden layers	two layers of 64
Optimizer	Adam

4.4 Deep Deterministic Policy Gradient (DDPG)

Deep Deterministic Policy Gradient (DDPG) is an off-policy actor–critic algorithm tailored for continuous action spaces [7]. In our implementation, we combine ideas from Deep Q-Networks (DQN) — such as experience replay and target networks — with a deterministic policy to stabilize learning in the stock-trading environment.

1. **Network Architecture.** We maintain four neural networks: an Actor and a Critic, each with a slowly updated target copy.

- *Actor* $\mu(s;\theta^{\mu})$:
  - Inputs the state vector s (normalized market features).
  - Passes it through two fully connected layers (128 units each, ReLU activations).
  - Outputs a raw action via a `tanh` activation, then scales and shifts it to match the environment's $[\text{act_low}, \text{act_high}]$ bounds.
- *Critic* $Q(s,a;\theta^{Q})$:
  - Processes s through one FC layer, concatenates the result with the (scaled) action a.
  - Feeds this combined vector into another FC layer (128 units, ReLU) and finally outputs a scalar Q-value.
- *Target Networks* $\mu'(s;\theta^{\mu'})$ and $Q'(s,a;\theta^{Q'})$:
  - Have the same architectures as the main networks.
  - Are updated slowly via soft updates (polyak averaging) to improve stability.

2. **Learning Procedure.** We train off-policy using a replay buffer and soft target updates:

- *Experience Replay:* We store transitions $(s_t, a_t, r_{t+1}, s_{t+1}, d_{t+1})$ in a buffer and sample random minibatches for training.
- *Exploration:* After an initial warm-up, we add Gaussian noise ($= 0.1$) to the Actor's raw `tanh` output, clip to $[-1,1]$, and then scale.
- *Critic Update:* We minimize the Bellman error

$$y_i = r_{i+1} + \gamma\,(1-d_{i+1})\,Q'\big(s_{i+1},\,\tilde{a}'_{i+1};\,\theta^{Q'}\big),$$

$$L_{\text{critic}} = \frac{1}{N}\sum_i \big(y_i - Q(s_i,\tilde{a}_i;\theta^{Q})\big)^2,$$

  where $\tilde{a}_i$ is the scaled action stored in the buffer and $\tilde{a}'_{i+1}$ comes from the Target Actor.
- *Actor Update:* We apply the deterministic policy gradient

$$\nabla_{\theta^{\mu}} J \approx \frac{1}{N}\sum_i \nabla_{\tilde{a}}\, Q\big(s_i,\tilde{a};\theta^{Q}\big)\big|_{\tilde{a}=\mu(s_i)}\, \nabla_{\theta^{\mu}}\,\mu(s_i;\theta^{\mu}),$$

  and minimize

$$L_{\text{actor}} = -\frac{1}{N}\sum_i Q\big(s_i,\mu(s_i;\theta^{\mu});\theta^{Q}\big).$$

  The Critic's parameters remain fixed during this update.
- *Soft Target Updates:* After each training step, we update the targets by

$$\theta' \leftarrow \tau\,\theta + (1-\tau)\,\theta',$$

  for both $\theta^{\mu'}$ and $\theta^{Q'}$.

3. **Hyperparameters.**

Table 4. Key DDPG Hyperparameters

Parameter	Value
Actor learning rate	1×10^{-4}
Critic learning rate	1×10^{-3}
Discount factor γ	0.99
Soft update rate τ	0.005
Replay buffer size	50,000
Minibatch size	128
Network hidden layers	(128, 128)
Exploration noise σ	0.1 (Gaussian)
Start-timesteps	1,000
Optimizer	Adam

5 Results & Analysis

We trained our DDPG agent on the historical data spanning our training period and then evaluated its performance over an unseen test set (January 1, 2019 to July 1, 2019). Rather than reproducing the code in this report, we summarize our procedure:

1. **Training Setup.** We initialized the `StockTradingEnv` with our preprocessed training DataFrame, set the initial portfolio value to \$1 000 000, and configured transaction costs, state and action spaces, and technical indicators. We then instantiated the DDPG agent with:

   - Discount factor $\gamma = 0.99$ and soft-update rate $\tau = 0.005$.
   - Actor/critic learning rates of 1×10^{-4} and 1×10^{-3}, respectively.
   - A replay buffer of size 50 000 and minibatches of 128.
   - Exploration noise $\sigma = 0.1$ after an initial 1 000 time-step warmup.

   We trained for 30 000 time steps, logging progress every 200 steps, and saved the final model for evaluation.

2. **Evaluation Procedure.** Using the same environment configuration but fed with our test DataFrame, we generated the agent's portfolio-value trajectory by running the trained policy in "prediction" mode. This yielded a time series V_t of daily account values over the seven-month test period.

3. **Benchmark Comparison.** To assess relative performance, we compared the agent's returns and risk metrics against a passive buy-and-hold investment in the Dow Jones Industrial Average (DJI) over the same dates.

6 Results & Analysis

We evaluate our trained DDPG agent on an unseen test period (January 1, 2019 to July 1, 2019) using the exact procedure from our notebook:

1. **Test Environment & Prediction.** We initialize a new `StockTradingEnv` with our test DataFrame and identical parameters to training, then generate the daily account-value series via

   `df_account_value` = `ddpg_agent.DRL_prediction(raw_env_ddpg_test)`.

   This produces a DataFrame of `date` and `account_value`, which we use for all subsequent calculations.

2. **Performance Metrics.** From `df_account_value`, we compute the daily return

   $$r_t = \frac{V_t}{V_{t-1}} - 1,$$

   where V_t is the portfolio value on day t. We then calculate:

   - *Cumulative Return (CR)*:
   $$\mathrm{CR} = \frac{V_{\mathrm{end}}}{V_{\mathrm{start}}} - 1.$$
   - *Sharpe Ratio (SR)*:
   $$\mathrm{SR} = \frac{\overline{r}}{\sigma_r}\sqrt{252},$$
   where $\overline{r}$ and σ_r are the mean and standard deviation of r_t, annualized over 252 trading days.
   - *Maximum Drawdown (MDD)*: the greatest peak–to–trough decline in V_t, measuring worst-case loss.
   - *Sortino Ratio (SoR)*:
   $$\mathrm{SoR} = \frac{\overline{r}}{\sigma_{\mathrm{down}}}\sqrt{252},$$
   where σ_{down} is the standard deviation of negative returns only.

3. **Benchmark Comparison.** To gauge our agent's value, we compare these four metrics against a passive buy-and-hold strategy on the Dow Jones Industrial Average (DJI) over the same test dates, computed in the same way from the DJI's daily closing prices.

Table **??** presents the numerical results for our DDPG agent and the DJI benchmark side by side.

6.1 Plots

Visualizations are crucial for understanding agent behavior and performance.

Placeholder: Training Curves Plot

Figure 2. Average episode return during training for REINFORCE, A2C, PPO, and DDPG agents. This shows the learning progress and convergence behavior.

Placeholder: Test Period Equity Curves Plot

Figure 3. Cumulative portfolio value over the test period (2019-01-01 to 2019-07-01) for each RL agent compared to the DJI benchmark (^{D}JI).$DDPGperformanceishighlighted.$

Placeholder: Final Portfolio Values Bar Chart

Figure 4. Bar chart comparing the final portfolio values achieved by each RL agent and the benchmark at the end of the test period.

Table 5. Summary of Performance Metrics on Test Data (2019-01-01 to 2019-07-01)

Strategy	Cumulative Return (%)	Annualized Sharpe Ratio	Max Drawdown (%)	Annualized Sortin
REINFORCE	[Value]	[Value]	[Value]	[Value]
A2C	[Value]	[Value]	[Value]	[Value]
PPO	[Value]	[Value]	[Value]	[Value]
DDPG	[Value]	[Value]	[Value]	[Value]
DDPG (Highlight)	**[Value]**	**[Value]**	**[Value]**	**[Value]**
DJI (^{D}JI)	[Value]	[Value]	[Value]	[Value]

Note: Values are placeholders based on expected outcomes.

6.2 Comparative Discussion

- **Algorithm Performance:** Based on the equity curves (Figure 3) and summary metrics (Table 5), the **DDPG** agent demonstrated the strongest performance, achieving higher cumulative returns and potentially better risk-adjusted returns (Sharpe/Sortino) compared to the policy gradient methods (REINFORCE, A2C, PPO) and the benchmark. REINFORCE, suffering from high variance, likely performed the poorest. A2C and PPO showed more stable learning (Figure 2) and moderate performance, possibly outperforming the benchmark but lagging behind DDPG.
- **Why DDPG Outperformed:** DDPG's potential success might stem from its ability to operate in a continuous action space (even if discretized later), allowing for finer-grained allocation decisions compared to the fixed buy/sell amounts in the purely discrete approaches. Its use of target networks and experience replay likely contributed to more stable learning in the complex, noisy financial environment, similar to DQN's advantages in other domains. The deterministic policy might also be beneficial if the optimal actions are indeed less stochastic given the rich state.
- **Effect of Sentiment Feature:** (Requires ablation study results not explicitly requested, but hypothesizing) An ablation study where agents were trained without the FinBERT sentiment scores would likely show a degradation in performance for the best agents (e.g., DDPG). This would suggest that news sentiment provides valuable, non-redundant information beyond technical indicators, helping the agent anticipate market movements or react more appropriately to news events, thus validating its inclusion in the state representation.
- **Limitations:**

- **Market Stationarity:** Financial markets are non-stationary; patterns learned in the training period (2016-2019) may not perfectly hold in the test period (2019) or future periods. Agents may need mechanisms for continual learning or adaptation.
- **Transaction Costs and Slippage:** While basic transaction costs were included, the simulation does not account for market impact or slippage (difference between expected and actual execution price), which can significantly affect profitability in live trading, especially with large orders.
- **Data Quality and Lookahead Bias:** Ensuring data alignment (e.g., news timestamps vs. market data close) is critical to avoid lookahead bias. Errors in data or sentiment analysis can negatively impact agent decisions. VIX and turbulence are market-wide, potentially simplifying the state space too much.
- **Survivorship Bias:** Using the current DJI30 constituents for historical analysis introduces survivorship bias, as it excludes companies that dropped out of the index due to poor performance.
- **Simplified Actions:** The discrete Buy, Hold, Sell or even the mapped continuous actions are simplifications of real-world order types and execution strategies.

7 Conclusion Future Work

This project successfully developed and evaluated a suite of reinforcement learning agents (REINFORCE, A2C, PPO, DDPG) implemented from scratch for the task of managing a portfolio of DJI30 stocks. By integrating market data, technical indicators, and news sentiment derived via FinBERT within a FinRL-based environment, we created agents capable of learning complex trading strategies.

Key Findings:

- Integrating diverse data sources, including news sentiment, provides a richer state representation potentially enabling RL agents to achieve better performance than using price/technical data alone.
- Actor-critic methods incorporating stability mechanisms like experience replay and target networks (DDPG) or policy constraints (PPO) generally exhibit more stable learning and superior performance compared to high-variance methods like REINFORCE in this noisy financial domain.
- The DDPG agent, despite needing adaptation for discrete actions, showed promising results, potentially outperforming stochastic policy gradient methods and the passive benchmark index on risk-adjusted return metrics during the test period.

While the results are encouraging, the limitations inherent in backtesting simulations necessitate cautious interpretation. Nonetheless, this work demonstrates the feasibility and potential of applying carefully implemented RL algorithms with multi-modal inputs to complex portfolio management tasks.

Future Work:

- **Advanced Agent Architectures:** Explore more sophisticated agent architectures, such as those incorporating attention mechanisms or transformers (e.g., Decision Transformer), which might be better suited to modeling long-range dependencies in financial time series and integrating heterogeneous data types more effectively.
- **Live Paper Trading and Robustness Testing:** Deploy the best-performing agent (e.g., DDPG or PPO) in a live paper trading environment (simulated trading with real-time data but no real money) to assess its performance under realistic market conditions, including latency, slippage, and potential regime shifts not fully captured in the historical backtest. Further robustness tests could involve varying transaction costs, data frequencies, and market conditions.

8 References

References

[1] *Placeholder citation for FinRL paper.*

[2] *Placeholder citation for Kaggle News Dataset.*

[3] *Placeholder citation for FinBERT paper.*

[4] *Placeholder citation for Sutton/Williams REINFORCE work.*

[5] *Placeholder citation for Mnih et al. A3C/A2C paper.*

[6] *Placeholder citation for Schulman et al. PPO paper.*

[7] *Placeholder citation for Lillicrap et al. DDPG paper.*